

Site-Determining Fragment Retention – Implementation & Test Plan

Streaming per-peptidoform gap coverage in BuildSpecLib, verified on a synthetic FASTA

Pioneer.jl – prepared for N. Wamsley

2026-07-03

1. Goal and the disk invariant

Retain, per peptidoform, top-N union {the highest-intensity in-gap fragment for each gap} so the library physically contains the ions that distinguish positional isomers (and, later, isomer decoys). See `site_determining_fragment_retention.md` for the algorithm and the worked VLSAGSPESIK example.

Hard constraint (yours): never materialize all fragments to disk. This is already satisfied by the current design and the plan preserves it. `filter_fragments! (::InstrumentAgnosticModel)` (BuildSpecLib `fragments/fragment_predict.jl:530`) receives one Koina batch in memory – “174 contiguous fragments per precursor,” grouped by ascending `precursor_idx` – and **returns only the top-N** (Pass 2, lines 598-614). `raw_fragments.arrow` is written *already-topN*. Our change adds at most `|gaps|` extra ions per precursor to that in-memory selection **before** the write. The full predicted series is never written.

2. Where it hooks (grounded)

Everything needed is already local to Pass 2 of `filter_fragments!`:

- per-precursor contiguous block (lines 600-614), `block = keep-rows sorted by -total`;
- `total[k]` (converted intensity, the ranking key);
- the parsed annotation `pa` per fragment – `pa.base_type` ('b'/'y'/'p'), `pa.frag_index` (ordinal), `pa.charge` – via `_agnostic_annotation!`;
- `ctx.sequences[pid]`, `ctx.mods[pid]`, `ctx.prec_len[pid]` (= peptide length L).

So the retention is computed from data already in hand, per precursor, with no new I/O and no cross-precursor state.

3. The change

Architecture: compute the gap sites at peptidoform *generation*, carry them to *prediction*.

The peptidoform generator is the authority on where mods can go – it already enumerates the acceptor sites (honouring fixed mods on acceptors, motif restrictions, min/max var-mods). Re-deriving “all S/T/Y” independently inside `filter_fragments!` risks diverging from what was actually enumerated. So we compute the potential-site set **S** once, at generation, stamp it per precursor, and carry it forward; the filter (and later the decoy generator) just consume it. **S** is a build-time intermediate – it rides the working precursor table, not the final `.poin`, so no library bloat.

3.1 At generation: compute and stamp the gap sites

Where variable-mod peptidoforms are enumerated (BuildSpecLib var-mod path – fillVarModStrings! / matchVarMods / chronologer_prep.jl), add:

```
compute_gap_sites(seq, enabled_var_mods, decoy_cfg) -> Vector{UInt8}    # sorted potential sites
```

$S = \text{sort}(\text{union over enabled var mods of their acceptor positions union (decoy on ? decoy-neighbour positions : \{\})})$. All peptidoforms of a base peptide share the same S (it is sequence-derived), so compute once per base peptide and stamp every peptidoform row with it. `decoy_neighbor_positions` must be the **same helper** the decoy generator uses (shared function), or reals won't retain the ions that distinguish the decoys (retention note, Section 5).

Carry S as a per-precursor field through Koina prep/parse into `AgnosticFragFilterCtx` (alongside the existing `ctx.sequences`, `ctx.prec_len`, `ctx.prec_mz`) as `ctx.gap_sites[pid]`.

3.2 At prediction: the pure retention function

```
gap_cover_indices(gap_sites, L, base_type, frag_index, total, block, topn_cut) -> Vector{Int}
```

Pure (no ctx, no I/O): given the carried `gap_sites` and one precursor's kept fragments (parallel arrays indexed by `block`, already sorted by descending `total`), returns the **extra** indices to retain beyond `block[1:topn_cut]`. It no longer needs the sequence or the acceptor logic – that lives at generation. Logic:

1. `gaps = [(gap_sites[j], gap_sites[j+1]) for j in 1:length(gap_sites)-1]`.
2. Fragment **cleavage position** `c = is_b ? frag_index : L - frag_index` (a y-ion of ordinal `m` cleaves after residue `L-m`). Fragment **crosses** gap `(a,b)` iff `a <= c < b`.
3. `block` is descending by `total`, so for each gap the **first** crossing block entry is that gap's highest-intensity in-gap fragment (per-peptidoform – this precursor's own spectrum).
4. Return the gap-winner indices **not already** in `block[1:topn_cut]`.

3.3 Wire into Pass 2

After computing `block` and `max_n` (line 608), append `gap_cover_indices(ctx.gap_sites[pid], ...)` to `final_order` for this precursor (in `block` order, so the emitted table stays intensity-descending – every extra ion sits below the top-N cut by construction, preserving the decode's rank=position invariant). One call per precursor; $O(|\text{frags}| * |\text{gaps}|)$, both small. If `gap_sites` is empty (no acceptors), it returns `[]` – a strict no-op.

4. Streaming / batching guarantees (your concern)

Per-peptidoform retention needs only the single precursor's fragments + its carried gap_sites – never its siblings. `gap_sites` is stamped at generation and identical for every peptidoform of a base peptide, but each peptidoform picks its own in-gap winners from its own spectrum. So:

- **No base-peptide co-location required.** A precursor and its positional-isomer siblings may sit in different Koina batches with no effect – each is self-contained.
- **The only requirement is that one precursor's fragments are not split across batches** – already guaranteed: Koina returns a fixed 174-fragment block per precursor and `parse_koina_batch` emits them contiguously per `precursor_idx`. Add a cheap assertion (each

`precursor_idx` block is whole and contiguous within the batch) to fail loudly if that ever changes.

- **Precursor sort is preserved.** `filter_fragments!` processes ascending `precursor_idx` and appends per-precursor blocks in order; we only reorder *within* a precursor (as top-N already does). The pre-prediction “intelligent” precursor sort that the fragment index depends on is untouched – we must not reorder precursors, only add fragments within each.

5. Config + no-op safety

- Gate: `build.site_determining_retention.{enabled, include_decoy_sites}`. Default `enabled=true` when the build has a variable mod with acceptor residues; `include_decoy_sites` follows the decoy config.
- **Non-phospho / no-acceptor builds are a strict no-op:** `acceptor_positions` empty -> `S` empty -> no gaps -> `gap_cover_indices` returns `[]` -> byte-identical to today. This must be an explicit test (Section 6.1) so we can guarantee zero regression to existing libraries.

6. Test plan (the important part)

Three levels, from fast+deterministic to end-to-end.

6.1 Unit – `gap_cover_indices`, hand-verified, no network

Drive it with the worked example so answers are checkable by hand.

Peptide VLSAGSPESIK (L=11, S at 3,6,9). Build parallel arrays for the b/y series with **controlled total values** so we can dictate the answer:

- Case A (phospho, `decoy_enabled=false`, `S={3,6,9}`, gaps (3,6),(6,9)): set the top-N (say N=10) to be shared ions (`b9,b10,y9,y10,...`) so the site-determining ions fall *outside* top-N; give `b4` the max total among gap-(3,6) crossers and `b7` the max among gap-(6,9) crossers. **Assert** `gap_cover_indices` returns exactly `{b4, b7}` -> retained = topN union `{b4,b7}`, and that the three isomers become pairwise-distinct on `{b4,b7}` (the (P,P)/(., P)/(.,.) table).
- Case B (per-peptidofrom): give isomer mod@3 a spectrum where `y7` is the strongest gap-(3,6) crosser, and mod@6 one where `b4` is. **Assert** they retain *different* ions for the same gap.
- Case C (decoy sites): `decoy_enabled=true` on GGGGGTGGK-style peptide (single acceptor, pro-line/glycine neighbours) -> assert the gap around the decoy neighbour is covered (`b_k/y`), i.e. real-vs-decoy becomes separable (mirrors retention-note Section 5).
- Case D (uncoverable gap): zero out every crosser of one gap -> assert that gap contributes nothing and is reported as uncoverable (for the ambiguity flag), without error.
- Case E (no-op): a peptide with no S/T/Y -> assert `gap_cover_indices == []`.

Verifiable because S, the gaps, and each ion’s cleavage are computed by hand from the sequence.

6.2 Pipeline – synthetic FASTA + MOCK predictor, deterministic, offline

Run the **real** BuildSpecLib fragment path (`filter_fragments! -> build_detailed_frags_from_raw -> compaction`) but replace the Koina call with a **stub predictor** that returns a *known* fragment table for the synthetic peptides (controlled intensities). This exercises the true code end-to-end with **provably known correct answers** and no network:

- Assert the built library’s stored fragments per peptidoform = `topN union {expected gap ions}`.
- Assert every pair of positional isomers has **distinct** stored fragment m/z sets, and that the distinguishing ion is a site-determining one we predicted by hand.
- Assert build **without** retention leaves at least one isomer pair identical (the regression the feature fixes) – a direct A/B.
- Assert the full 174-fragment series was **never** written (inspect the intermediate is `topN+gaps`).

Mechanism: dispatch a test-only predictor type (or inject the raw batch DataFrame directly ahead of `filter_fragments!`), so the stub is the single seam and the rest is production code.

6.3 Integration – synthetic FASTA + real Koina (network, manual/tagged)

The offline tests fix the logic; this confirms real Prosit behaves. Build a small library from the synthetic FASTA against live Koina and assert **structurally** (intensity-independent): for each peptidoform and each gap, a crossing ion is stored *unless* Prosit predicts nothing in that gap (then it is flagged uncoverable). Tagged like the existing SearchDIA integration test, not in fast CI.

6.4 Synthetic FASTA design (checked-in test fixture)

A handful of short designed proteins whose tryptic peptides are the known-answer cases:

- `>iso3 ...VLSAGSPESIK...` – three acceptors at 3/6/9 (the worked example; multi-isomer).
- `>pro1 ...GGGGTGGPGGK...` – single acceptor T with a proline neighbour (phospho-proline decoy distinguishability).
- `>adj ...AASSAAK...` – adjacent acceptors S4,S5 (the hard uncoverable/near-collinear case).
- `>none ...LLEELLK...` – no S/T/Y (no-op regression guard).

Peptides kept short so the full peptidoform set and the correct retained ions are enumerable by hand. Fixture lives in `test/UnitTests/fixtures/`; the truth (per peptide: S, gaps, expected gap ions) is a small companion table asserted against.

7. Risks / edge cases

- **y-ion cleavage convention** – `c = L - frag_index`; unit-test both b and y explicitly (easy off-by-one).
- **Charge / isotope variants** – the same cleavage yields b/y at multiple charges (and isotopes); all count as in-gap candidates, and the `max-total` winner may be a 2+ ion. Test a mixed-charge case.
- **Decode rank=position invariant** – extra ions must be appended *after* the top-N (they are, being below the cut); assert the emitted block stays `total`-descending.
- **Uncoverable gaps** – surface per-peptidoform (Section 6.1 Case D) into the localization ambiguity flag rather than silently dropping.
- **Site-set parity with the decoy generator** – `decoy_neighbor_positions` here and the decoy site choice in `generate_isomer_decoys!` must be the *same* function; share one helper.
- **Multi-mod generalization** – `acceptor_positions` = union over enabled variable mods; near-isobaric mod masses can still leave a boundary ambiguous (degrades to the uncoverable-gap case). Phospho-only first.

8. Implementation steps

1. **Pure functions + unit tests (offline, land first):** `gap_cover_indices(gap_sites, ...)` plus `acceptor_positions` / `decoy_neighbor_positions` / `compute_gap_sites` (the last shared with the decoy generator) + the Section 6.1 hand-verified tests. This is the whole algorithm, testable with zero build machinery.
2. **Generation + carry:** call `compute_gap_sites` at var-mod enumeration, stamp a per-precursor `gap_sites` field, and thread it through Koina prep/parse into `AgnosticFragFilterCtx.gap_sites`. Test: every peptidoform of a base peptide carries the same `S`, and `S` matches the hand-computed set for the synthetic fixtures.
3. **Wire into filter_fragments! Pass 2:** consume `ctx.gap_sites[pid]`; config gate; the batch-contiguity assertion; the no-op (empty `gap_sites`) regression test.
4. **Synthetic FASTA fixture + mock-predictor pipeline test (6.2)** – the deterministic known-answer end-to-end (retained set == top-N union expected gap ions; isomers distinguishable; full series never written).
5. Optional real-Koina integration test (6.3).
6. Only then build the phospho standards library with retention on and re-run the FLR harness to confirm the retained site-determining ions move the residual adjacent-site errors.